

Reading Without Extra Lines

Solving the "Double Spacing" problem when printing file contents in Python.

Prepared for Python Learners



The Challenge

Goal

We have a file `examples.txt`. We need to write a program that prints its content to the console exactly as it looks in the file.

- ✓ **Input:** A standard text file.
- ✓ **Issue:** Default printing often creates large gaps between lines.
- ✓ **Requirement:** "No extra blank lines" in the output.



Why does it happen?

The Collision of Newlines

1. **The File:** Each line in a text file typically ends with a hidden `\n` character.

2. **The Print Function:** Python's `print()` function adds its own `\n` at the end by default.

Result: `\n + \n =` Two blank lines.

The Bug in Action

```
line_from_file = "Hello\n"  
print(line) → adds "\n"
```

Output:
"Hello\n\n"

Expected vs. Actual



File Content

Line 1

Line 2

Line 3



Standard Print

Line 1

Line 2

Line 3



Correct Output

Line 1

Line 2

Line 3

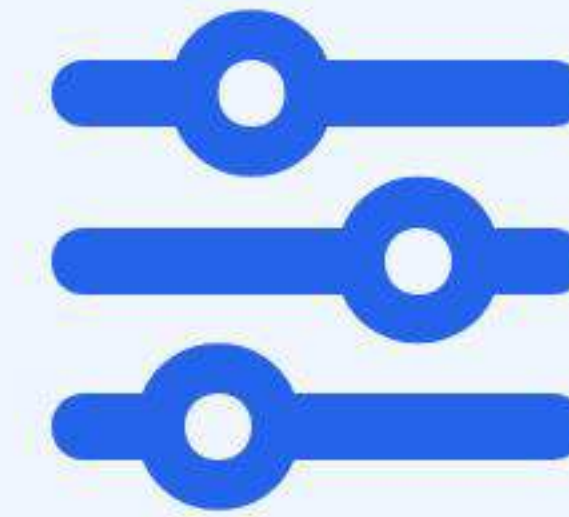
Solution 1: Modifying Print

The `end` Parameter

Python's `print()` function has a secret weapon: the `end` parameter.

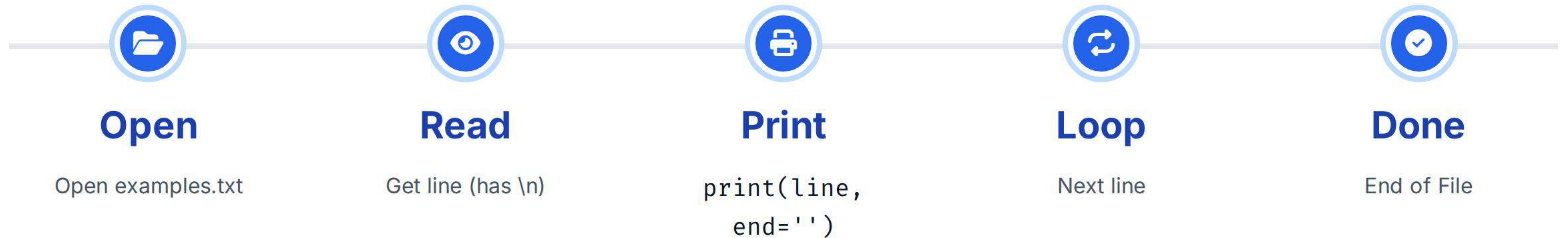
By default, `end='\n'`. We can change this to an empty string `end=' '` to stop Python from adding that extra newline.

This tells Python: "Just print the text I gave you, don't add anything else."



Configuration Change

Flowchart: Using end=''



Code: Using end=''

```
def print_exact_content():
    try:
        with open('examples.txt', 'r') as file:
            for line in file:
                # Print the line, but disable the
                # automatic newline at the end
                print(line, end='')

    except FileNotFoundError:
        print("File not found")

print_exact_content()
```

Code Breakdown

- `for line in file:` Reads line-by-line. Each `line` variable contains the text *plus* the newline character from the file.
- `end=''`: Replaces the default newline with nothing.

Solution 2: Cleaning the Data

The `strip()` Method

Instead of changing how we print, we can change the data itself.

The `.strip()` string method removes whitespace from both the beginning and end of a string, including the pesky `\n`.

If we strip the newline, we can let `print()` add its own newline back in. It balances out!



Data Cleaning

Flowchart: Using strip()



1. Input

Line = "Hello\n"



2. Clean

```
line.strip()  
Result = "Hello"
```



3. Output

```
print() adds \n  
Final: "Hello\n"
```


Code: Using strip()

```
def print_stripped_content():
    try:
        with open('examples.txt', 'r') as file:
            for line in file:
                # Remove leading/trailing whitespace
                # including the newline character
                clean_line = line.strip()

                # Print normally (adds its own newline)
                print(clean_line)

    except FileNotFoundError:
        print("File not found")

print_stripped_content()
```

Code Breakdown

- `line.strip()`: Removes the original newline from the file data.
- `print()`: Adds a brand new newline character.
- **Result:** 1 - 1 + 1 = 1 newline.

Comparing Approaches

Feature	Solution 1 (end='"')	Solution 2 (strip())
Data Integrity	Exact (Keeps indentation)	Altered (Removes indent)
Whitespace	Preserves spaces	Deletes edges
Logic	Prints raw input	Cleans input first
Best Use	Exact file reproduction	Processing data/tokens

Critical Difference: Indentation

What if lines are indented?

If your file contains Python code or indented lists:

- ✓ `end= ''` keeps the spaces. The indentation remains.
- ✓ `strip()` **deletes** the spaces. The indentation is lost!

File Input: " indented"

```
end= ' ': " indented"  
strip(): "indented"  
(Structure Lost!)
```